plan-failure-bench: A Machine-Checkable Benchmark of How Language Model Planners Fail

Munawar Kazmi
munawarsaeedkazmi1@gmail.com

Working paper, under active development. July 2026.
Repository: `https://github.com/munawarkazmi/plan-failure-bench`

Abstract

Success rates blur the distinction between a language model that refuses every task and one that walks into every trap. plan-failure-bench measures how language model planners fail, not merely how often. Each of 60 instructions over two symbolic household environments either admits a valid plan or plants exactly one trap from a six-way taxonomy: unreachable goal, missing capability, ambiguous referent, precondition trap, sequencing trap, or constraint trap. Models answer in a small JSON action language with explicit infeasible and clarify responses, so trap detection is machine-checkable. A deterministic checker, differentially tested against an independent PDDL toolchain, assigns exactly one verdict per response; every planted label carries a mechanical proof obligation re-run in continuous integration; and every instruction also exists in a semantically obfuscated condition applied as an invertible renaming, so the checker only ever scores the canonical world. The headline artefact is the confusion matrix between planted trap and observed verdict. Early results on four models are reported as counts and hypotheses: models fail in distinct, stable ways; a 70B model's planning judgement survives semantic obfuscation essentially intact, an apparent execution collapse under our first token scheme having been caught and retired by the obfuscation's own versioning; over-refusal tracks surface semantics; for the smaller reasoning-generation model, unreachability detection survives obfuscation only where the isolation is stated rather than left to be inferred from topology; and the first frontier reasoning model clears house_01 in both conditions with identical ideal diagonals, then repeats every headline count on the second environment in both conditions, the same single sequencing seed failing identically in each, so semantic removal changes nothing this benchmark can measure for it, inferred topology included; that bounds every failure finding here to smaller and non-reasoning models. This is a working paper; the results tables are generated directly from the committed run records.

1 Introduction

When a household robot is driven by a language model planner, the dangerous failures are not the clumsy plans but the confident ones: the model that plans a route through a room it was told never to enter, or fetches one of two identical cups without asking which. Before asking whether a language model plans well, a benchmark should ask whether the model notices when planning is the wrong response altogether.

plan-failure-bench makes that question decidable. Its design contributions:

1. **Ground truth decidable by construction.** The world model is a finite set of ground atoms with no numerics, so reachability questions are decidable and every label is provable.

2. **Proof obligations in continuous integration.** Every seed's label carries a mechanical proof (a validating reference plan, an unreachability proof from a sound abstraction, a capability grant that flips infeasibility, or exhaustive referent bindings), re-proved on every commit by 548 tests.

3. **Refusal and clarification as first-class answers.** The response language contains `infeasible(reason)` and `clarify(candidates)` terminals, so detection is scored mechanically and always reported alongside the paired false positive count on feasible instructions.

4. **A semantically obfuscated condition.** Content words are renamed by a versioned bijection applied to the prompt and inverted on the response, separating state tracking from lexical pattern matching while the checker scores only the canonical world.

5. **Two structurally contrasting environments.** The second environment varies topology and constraint structure specifically to test whether findings from the first are artefacts of one map.

6. **Open, replayable records.** Every run is committed as one JSON record per seed, and both scoring policies re-score offline without model calls.

2 Related work

Draft status: the bibliographic details of every citation below and the specific findings attributed to them were checked against the cited papers on 25 July 2026, with two entries added and checked on 30 July 2026; the verification log at the end of the section records each check. A final sweep for work newer than 30 July 2026 precedes submission.

Evaluating LLM planning. PlanBench established systematic evaluation of LLM planning over symbolic domains and its successors have repeatedly found generation weak [15, 16]. The same line introduced Mystery Blocksworld, obfuscating a familiar domain to separate reasoning from lexical recall, and found that plan generation collapses; plan-failure-bench inherits that manipulation but applies it as a versioned bijection and asks a broader question, namely which *judgements* survive semantic removal, not only whether generation does. Unsolvability recognition itself is not new: PlanBench ships unsolvable instances scored by true and false negative rates, and Valmeekam et al. [17] report o1-preview explicitly recognising 27% of unsolvable Blocksworld instances, 16% under randomised obfuscation, while wrongly declaring 11.5% of solvable obfuscated instances impossible. plan-failure-bench differs in breadth and pairing rather than in raising the question: six trap families under one response protocol, detection always paired with false positives, and per-seed proof obligations. Pipeline alternatives such as LLM+P hand the search problem to a classical planner and use the model only for formalisation [7]; plan-failure-bench instead measures the model as the planner, because the failure modes of that configuration are what the benchmark exists to characterise.

Feasibility, ambiguity, and safety as separate literatures. Each trap family here has a single-family predecessor. Plancraft includes intentionally unsolvable crafting tasks scored mechanically: given an explicit impossible action, its best model reaches F1 0.71 at declaring impossibility, and models become overeager, trading success on solvable tasks for refusals [2]. AmbiK pairs every ambiguous kitchen instruction with an unambiguous counterpart and scores whether detection methods can tell the two apart [4], and the asking-for-help line, KnowNo's calibrated uncertainty and CLARA's command classification, trades help requests against success guarantees [12, 10]. Two 2026 benchmarks make the pairing suite-wide in the

tool-use setting: TREK pairs 533 feasible travel-planning tasks with 267 provably infeasible ones whose causes are typed (route, entity, budget), scores with deterministic rules and no LLM judge, and grades a bare refusal strictly below one that names the right cause [11]; AgentAbstain pairs every should-abstain task with a should-act variant differing in exactly one perturbation across eight scenarios, so that no constant policy can exceed 50% paired accuracy, though an LLM judge scores the terminal response [8]. Paired or trade-off measurement therefore exists within single families and, in tool-use domains, suite-wide, including the false impossibility claims measured by Valmeekam et al. [17]; plan-failure-bench carries the pairing into embodied planning across six trap families under one protocol. SafeAgentBench shows embodied agents complying with hazardous instructions and documents the over-refusal trade-off directly, a safety module raising rejection of safe tasks alongside hazardous ones [19]; HomeBench evaluates valid and invalid instructions in smart home device control, where even frontier models fail badly on invalid multi-device cases [6]. XSTest documents the chat-model analogue of over-refusal, driven by surface features of safe prompts [13], and Semantic Denial of Service attacks show robot refusal triggering on injected surface-level safety phrases regardless of physical state [14], adversarial evidence of the same anchoring our obfuscation manipulation measures. The nearest neighbour to our aims is the Yes-Man Syndrome benchmark [18], which measures abstention by embodied agents over ambiguous, physically infeasible, false-premise, and sensing-unresolvable instructions. Its instructions are generated by deterministic constraint derivation, but its scoring uses an LLM judge, it measures only whether agents abstain when they should, with no false positive column for valid instructions, and it has no semantic obfuscation condition; its categories all target abstention, while half of plan-failure-bench's taxonomy consists of feasible instructions carrying planted execution traps, which is what makes the planted versus observed confusion matrix possible. To our knowledge no prior benchmark manipulates surface semantics while measuring refusal, the manipulation behind our finding that refusal behaviour is anchored to surface semantics in opposite directions for different models.

Error taxonomies and ground truth. The Embodied Agent Interface catalogues fine-grained error types across goal interpretation, decomposition, sequencing, and transition modelling [5]. plan-failure-bench differs in direction: instead of classifying observed errors after the fact, it plants exactly one labelled trap per instruction and reports the confusion matrix between planted and observed, asking whether models fail as predicted. Doing that credibly requires ground truth that defends itself, which places this work in the plan-validation tradition of VAL [3]: verdicts come from a deterministic checker differentially tested against an independent PDDL executor, pyperplan [1], labels are proofs re-run in continuous integration, and no human or model judgement appears anywhere in the scoring loop. Plancraft also scores mechanically inside its crafting environment and TREK's evaluator is deterministic rules over its travel knowledge base, while RoboBench delegates plan feasibility scoring to a multimodal model acting as a world simulator [9] and the Yes-Man benchmark scores with an LLM judge. What no benchmark above combines is judge-free scoring with per-seed proof obligations re-run in continuous integration and differential testing of the checker itself.

Verification log for this section; items (1) to (6) discharged 25 July 2026, item (7) on 30 July 2026. (1) *Done:* bibliographic details of every entry confirmed against public records; Plancraft's venue upgraded to COLM 2025 and AmbiK's to ACL 2025 in the process. (2) *Done:* Plancraft's paper body read; the F1 0.71 figure, the overeager declaration trade-off, and its mechanical scoring are stated above from the body, not the abstract. (3) *Done:* unsolvability recognition under obfuscation is prior art and the section says so with figures confirmed against the paper body (17); the specific dissociation we observe, detection surviving obfuscation while execution collapses within one model, remains unlocated in prior work. (4) *Done:* paired measurement exists within single families and

the text credits each instance, AmbiK's counterparts, KnowNo's help budget, Plancraft's overeager declarations, and the false impossibility claims of 17; our pairing claim is its systematic extension across all six families. (5) *Done as far as searching can resolve it:* no embodied refusal benchmark manipulating surface semantics was found; the claim stands as "to our knowledge". (6) *Done:* the sweep's reading list is fully read. Yes-Man scores with an LLM judge and reports no false positive column on valid instructions, confirmed from the paper body; HomeBench and Semantic Denial of Service are now cited; RoboBench is cited as a contrast on judge-free scoring. (7) *Done, 30 July 2026:* the sweep for work newer than 25 July found TREK (submitted 29 July 2026), whose feasible–infeasible split, typed causes, judge-free deterministic scoring, and cause-naming requirement are stated above from the paper body; the same sweep surfaced AgentAbstain (11 July 2026), which the 25 July reading list had not reached, and its paired-task construction, paired accuracy definition, and LLM-judge terminal scoring are likewise stated from the body. One standing item remains: a final sweep for work newer than 30 July 2026 before submission.

3 Benchmark design

3.1 World model

An environment is a set of rooms connected by doors (open, closed, or locked), items with categories and properties located in rooms, a robot with a capability profile over a fixed action vocabulary (goto, open, close, pick, place, and optionally unlock), a single-slot gripper, and trajectory invariants that must hold in every state, not only at the end. Two invariant kinds exist: never_enter (the robot must never be in a room) and never_hold_in (the robot must never be in a room while holding an item with a given property). Invariant text is shown to the model verbatim, so constraint seeds test planning, not telepathy. State is a finite set of ground atoms over a fixed object universe; there are no numerics, so the state space is finite and reachability is decidable.

3.2 Task language

Models receive a fixed rendering of the environment and one natural language instruction, and must answer with exactly one JSON object: a `plan` (a list of actions), `infeasible` with a reason from {unreachable, missing_capability, constraint}, or `clarify` with candidate referents. The strict scoring policy, the headline, requires exactly a response-shaped JSON object; a documented lenient policy re-scores stored records offline by extracting the first response-shaped JSON object, separating format discipline from planning ability.

3.3 Trap taxonomy and proof obligations

Each instruction either admits a valid plan or plants exactly one trap. Labels carry these proof obligations, re-proved on every test run:

- **valid**: a reference plan validates under the checker and under independent PDDL execution, and no infeasibility proof exists.
- **unreachable_goal**: provably unreachable under a sound over-approximating abstraction, and still unreachable with the full action vocabulary granted.
- **missing_capability**: provably unreachable under the robot's profile, not provable once a named capability is granted, and a capability reference plan validates in the augmented environment.

- **ambiguous_referent**: at least two same-category bindings, each with a validating reference plan and no infeasibility proof; the expected answer is clarify with exactly the category's members.
- **precondition_trap** and **sequencing_trap**: the goal is feasible (reference plan validates); a decoy plan documenting the intended error produces exactly the declared trap verdict.
- **constraint_trap**: either feasible with a decoy that executes fully, achieves the goal, and silently breaches an invariant, or provably infeasible solely because of an invariant.

3.4 Environments

house_01: seven rooms, six doors, ten items, two never_hold_in invariants (each property carried by exactly one item), a doorless cellar whose isolation the environment description states outright, and a locked store room behind the withheld unlock capability. It contains one cycle (kitchen, hallway, living room), passable only through a closed door.

office_01: nine rooms, eight doors, eleven items, built so that its structure contrasts with house_01 rather than repeating it. A five-room ring reachable through open doors makes route choice pervasive. A never_enter room sits on the ring, so the short route between reachable rooms can silently violate an invariant by movement alone. A never_hold_in property is carried by three items rather than one. A two-room annex is disconnected from the rest of the map, but both its rooms have doors, so no "has no doors" statement appears and the isolation must be inferred from the connection list. Ambiguous referent candidates can sit in different rooms, and one decoy traps the single-slot gripper itself. Both suites plant the same label distribution (9 valid, 4 unreachable, 3 missing capability, 3 ambiguous, 4 precondition, 3 sequencing, 4 constraint), keeping confusion matrix columns comparable across environments.

3.5 Obfuscated condition

Semantic content words (rooms, doors, items, categories, properties, actions, door states) are replaced by deterministic nonsense tokens; structural vocabulary (room, door, item, gripper) and the response protocol are retained. The renaming is a bijection applied to the assembled prompt and inverted on the model's raw response before checking, so the checker, the seeds, and every label proof operate only on the canonical world and no second world model exists to drift. Version 1 tokens were confusable enough that model copy errors were indistinguishable from hallucination; version 2 tokens guarantee unique opening syllables and pairwise edit distance of at least three. Every record stores the obfuscation version it was produced under.

4 Ground truth guarantees

The checker simulates each plan step by step and assigns exactly one verdict: valid, precondition_violation, constraint_violation, goal_not_achieved, hallucinated_entity, unavailable_action, or malformed, plus the two terminals. It is differentially tested against pyperplan over hand-written trap plans and hundreds of seeded random and guided plans, with first-failing-step agreement required. Trajectory invariants are deliberately excluded from the PDDL compilation, since STRIPS has no trajectory constraints; the differential oracle covers executability and goal satisfaction, and invariant semantics are covered by direct tests. Infeasibility labels are proofs from a per-literal over-approximating abstraction whose direction is sound: every real trajectory has an abstract counterpart, so goals the abstraction cannot

Model	Environment	Condition	Note	Format failures	Traps detected	Exact reasons	False positives	Valid solved
Llama 3.3 70B	house_01	plain		18/30	9/13	6	3/17	5/9
Llama 3.3 70B	house_01	obfuscated (v1)	superseded	23/30	11/13	9	1/17	1/9
Llama 3.3 70B	house_01	obfuscated (v2)		26/30	10/13	8	0/17	5/9
Qwen 2.5 7B	house_01	plain		3/30	2/13	0	0/17	2/9
Qwen 2.5 7B	house_01	obfuscated (v1)	superseded	5/30	3/13	1	0/17	1/9
Qwen 2.5 7B	house_01	obfuscated (v2)		10/30	3/13	1	3/17	1/9
Gemini 3.1 Flash Lite	house_01	plain		0/30	12/13	8	4/17	6/9
Gemini 3.1 Flash Lite	house_01	obfuscated (v2)		0/30	7/13	6	1/17	5/9
Gemini 3.6 Flash	house_01	plain		0/30	13/13	10	0/17	9/9
Gemini 3.6 Flash	house_01	obfuscated (v2)		0/30	13/13	10	0/17	9/9
Llama 3.3 70B	office_01	plain		24/30	9/13	5	1/17	2/9
Llama 3.3 70B	office_01	obfuscated (v2)		27/30	7/13	5	1/17	2/9
Qwen 2.5 7B	office_01	plain		4/30	1/13	0	0/17	2/9
Qwen 2.5 7B	office_01	obfuscated (v2)		13/30	3/13	2	2/17	3/9
Gemini 3.1 Flash Lite	office_01	plain		1/30	10/13	6	7/17	4/9
Gemini 3.1 Flash Lite	office_01	obfuscated (v2)		1/30	5/13	4	2/17	2/9
Gemini 3.6 Flash	office_01	plain		0/30	13/13	10	0/17	9/9
Gemini 3.6 Flash	office_01	obfuscated (v2)		0/30	13/13	10	0/17	9/9

Table 1: Every complete committed run. Format failures are strict-policy malformed responses out of 30; all other columns use lenient extraction. Traps detected counts the 13 seeds per suite whose expected answer is a terminal, and is never read without the paired false positives on the 17 feasible instructions. Exact reasons counts detections whose reason code or candidate set was exactly right.

reach are truly unreachable, and reachable-looking goals are proved feasible constructively by reference plans, never by the abstraction.

5 Experimental setup

Four models, all under a fixed prompt at temperature 0 with one decode per seed: Llama 3.3 70B (hosted), Qwen 2.5 7B Instruct (local), Gemini 3.1 Flash Lite (hosted), and Gemini 3.6 Flash (hosted; a reasoning generation model, with both house_01 columns and the plain office_01 column complete and the obfuscated office_01 column accumulating on daily free-tier quota). The three original models cover both suites in both conditions under version 2 tokens. The superseded version 1 obfuscated runs of Llama and Qwen are retained and marked in the tables, because the artefacts they exposed are part of the methodology's history.

6 Results

All numbers below are single-decode counts under the lenient extraction policy, with strict format failures reported per run; at $n = 30$ per condition they are hypotheses, not claims. Table 1 is generated directly from the committed records.

Models fail in distinct, stable ways. Llama detects most traps but violates the response format in 18 of 30 plain responses, wrapping correct JSON in prose. Qwen is format-disciplined and almost never refuses: zero false positives in plain on both environments, with nearly every trap ending in an observed precondition violation. Gemini Flash Lite is format-perfect on house_01 with near-ceiling detection, but solved none of house_01's seven

Model	Environment	Token scheme	Hallucinated entities
Llama 3.3 70B	house_01	v1	4
Llama 3.3 70B	house_01	v2	0
Qwen 2.5 7B	house_01	v1	15
Qwen 2.5 7B	house_01	v2	1
Gemini 3.1 Flash Lite	house_01	v2	0
Gemini 3.6 Flash	house_01	v2	0
Llama 3.3 70B	office_01	v2	1
Qwen 2.5 7B	office_01	v2	0
Gemini 3.1 Flash Lite	office_01	v2	0
Gemini 3.6 Flash	office_01	v2	0

Table 2: Hallucinated-entity verdicts across obfuscated runs. The v1 token scheme manufactured spurious hallucinations by confusability.

ordering traps and refused more feasible instructions than any other model. Each profile replicated in direction on the second environment; Llama's office_01 plain run kept detection at exactly its house count (9 of 13) while format failures worsened to 24 of 30 and valid-seed success fell to 2 of 9.

A finding retired by the methodology's own versioning. Under version 1 tokens, Llama appeared to show a dissociation: detection surviving obfuscation while valid-seed success collapsed from 5 of 9 to 1 of 9. The version 2 rerun refutes the collapse: detection holds (9 of 13 to 10 of 13, false positives 3 of 17 to 0 of 17) and execution holds as well, 5 of 9 in both conditions on house_01 and 2 of 9 in both on office_01, with Llama's hallucinated-entity count falling from 4 under version 1 to 0. The execution collapse was token confusability damage. What survives is simpler and still contrary to a pure pattern-matching account: for this model, planning judgement and execution are both essentially unimpaired by semantic removal.

Over-refusal tracks surface semantics. Gemini's false positives fell from 4 of 17 to 1 of 17 under obfuscation on house_01 and from 7 of 17 to 2 of 17 on office_01. Qwen moved the other way: from zero false positives in plain to 2 and 3 of 17 under obfuscation, and its only exact-reason detections anywhere are constraint seeds under obfuscation, instructions it silently complied with in plain English.

For the smaller reasoning-generation model, stated facts survive obfuscation and inferred topology does not. On house_01, where the isolated room's doorlessness is stated in the environment description, Gemini 3.1 Flash Lite's unreachability detection survived obfuscation at 4 of 4 with exact reasons. On office_01, where the annex's isolation must be inferred from the connection list, its unreachability detection fell from 3 of 4 to 1 of 4 under obfuscation, the survivor being a nonexistent object rather than any topology seed. The frontier model is not so bounded: it detects all four office unreachable seeds in both conditions, annex seeds included with exact reasons, so the stated-versus-inferred split separates the two reasoning-generation models rather than describing them both.

The methodology has caught two of its own artefacts. Under version 1 tokens, Qwen showed 15 hallucinated-entity verdicts on house_01 (version 2: 1) and Llama showed 4 (version 2: 0) alongside the spurious execution collapse retired above (Table 2). Records carry their obfuscation version, so generations of results never mix silently, and the superseded runs remain visible in Table 1.

Sampling stability: the small model's profile is not a decoding artefact. Five decodes per seed at temperature 0.7 (Qwen, plain, house_01) produced the identical lenient verdict on 26 of 30 seeds. Detection was exactly binary: the two object-level unreachable seeds were detected in all five samples (with the same wrong reasons), the remaining eleven trap

seeds in none of their 65 decodes, and none of the 85 feasible-seed decodes produced a refusal. The never-refusing profile survives sampling. The obfuscated cell (same protocol) is noisier, 14 of 30 seeds fully stable, yet its structure repeats: the nonexistent-object and inexpressible-verb seeds and the compound constraint seed are detected in all five samples, and one valid seed is refused in all five, so the obfuscation-induced refusal mode is reproducible rather than decode luck. The office_01 cells complete this model's grid and replicate both patterns: the plain cell is again stable (24 of 30 seeds, zero refusals across 85 feasible decodes) and the obfuscated cell again destabilises (17 of 30) around a reproducible refusal core, with the compound greasy constraint seed detected with the exact reason in all five samples. Sampling at temperature 0.7 also reintroduces token copy errors that the version 2 edit distance guarantee suppresses at temperature 0: 13 of 150 house_01 and 11 of 150 office_01 obfuscated decodes produced a hallucinated-entity verdict, against 1 and 0 of 30 at temperature 0.

Sampling the wrapper model: instability is detection flicker, not a stable wrong answer. The same protocol applied to Llama (plain, house_01) gives 19 of 30 seeds the identical lenient verdict in all five samples, with strict format failures of 17, 18, 17, 18, and 16 of 30 per sample and lenient extraction leaving 1, 4, 1, 0, and 1 malformed. Seven of the 13 trap seeds are detected in all five samples, five in some, one in none; two of the 17 feasible seeds are refused at least once, one in every sample. Where Qwen's sampled verdicts freeze, Llama's flicker at the detection boundary: three infeasibility seeds mix `terminal_infeasible` decodes with observed `precondition_violation` decodes, the same seed detected in one sample and executing into the missing precondition in the next. The silent-violation constraint seed splits two to three: two decodes take the planted bait, a fully executing plan that breaches the constraint, and three refuse the feasible instruction, every case mechanically separated by the checker.

The frontier model clears house_01 in both conditions, and the rows are identical. Gemini 3.6 Flash, plain and fully obfuscated: no format failures, 13 of 13 traps detected (10 exact reasons), zero false positives, and 9 of 9 valid seeds solved in each condition, including all seven ordering traps that defeated every other model and the compliant route on the silent-violation constraint seed, chosen even when the constraint concerned nonsense words in nonsense rooms. Both confusion matrices are the ideal diagonal. For this model on this environment, the benchmark's central manipulation returns a clean answer: its planning judgement is state tracking, not lexical pattern matching. The micro-exceptions are diagnostic wording, not judgement: under obfuscation its one exact capability diagnosis reverted to unreachable while its unreachable reasons became 4 of 4 exact. Every failure claim in this paper is therefore bounded to smaller and non-reasoning models.

The frontier model's second environment: identical headline counts in both conditions, one repeated off-diagonal cell. Gemini 3.6 Flash on office_01, plain and fully obfuscated alike, repeats every headline count of its house_01 rows: no format failures, 13 of 13 traps detected (10 exact reasons), zero false positives, 9 of 9 valid seeds solved. Its unreachability detection covers all four office seeds in both conditions, including the two annex seeds whose isolation is never stated and must be inferred from the connection list, so for this model topological inference itself survives semantic removal, where the smaller reasoning-generation model kept only the stated fact. Neither office matrix, however, is the ideal diagonal: in each condition the same sequencing trap, whose stated order closes the robot's own route so that only a reordered plan achieves the goal, produced a plan satisfying one of two goal conjuncts, `goal_not_achieved`, the model's only non-valid verdicts on feasible seeds across its four committed columns. Aggregates hide it: the failure sits on a feasible seed outside the valid label, so every column of the summary table stays perfect while the matrix does not. And the house_01 pattern repeats in full: the two office rows are identical to each other, blemish included.

Format discipline is the model's habit, not the prompt's artefact. Three prompts over the same seeds (Llama, plain, house_01): the canonical prompt yields 18 of 30 strict format failures, a bare only-JSON variant 12 of 30, and a format-contract-last variant 15 of 30, so no wording cures the prose-wrapping. The bare variant backfires where it seems to help: 8 of its 30 responses contain no recoverable JSON at all, against 1 of 30 under the canonical prompt. Lenient planning metrics barely move across prompts (detection 8 to 10 of 13, false positives 2 to 3 of 17), which is precisely the separation the two-policy scoring exists to provide: format discipline is prompt-sensitive, planning conclusions are not.

The capability distinction still defeats every model, and the crack in it now survives obfuscation. The capability-grant seeds, where the suite proves a goal is sealed by a missing capability rather than topology, have never received the exact diagnosis in any run: even the frontier model calls them unreachable, on office_01 as on house_01, in every condition. The only exact missing-capability reasons ever produced came from Gemini 3.6 Flash on the two inexpressible-verb seeds, house_01 mopping and office_01 photocopying: three exact diagnoses across eighteen committed fixed-prompt columns. The house_01 diagnosis appeared in plain language and reverted to unreachable under obfuscation; the office_01 diagnosis held in both conditions, the first exact capability reason to survive semantic removal.

7 Limitations

The scope is four models (Llama 3.3 70B, Qwen 2.5 7B Instruct, Gemini 3.1 Flash Lite, Gemini 3.6 Flash), two environments, and 30 seeds per condition. Counts, not rates: every table cell is one decode at temperature 0, so no statistical treatment is attempted yet. The $k = 5$ consistency experiment covers Qwen's four cells (both environments, both conditions), where it found the single decodes representative in direction, and Llama's plain house_01 cell; the remaining cells are unsampled. Two environments in one domain family (indoor rooms, doors, portable items) bound the generality of every claim. Prompt sensitivity is quantified for one model only (Llama, plain, house_01); the other models' format behaviour under prompt variation is unmeasured. The frontier model's ideal diagonals are a house_01 result: its office_01 columns repeat the headline counts in both conditions but not the perfect matrix. The lenient policy is a fixed syntactic extraction rule, not a semantic judge, by design. All hosted models ran on free-tier quotas, which selected the model list.

8 Future work

The sampling protocol ($k = 5$ decodes per seed at temperature 0.7 into separate record files, reported as per-seed verdict consistency) has covered Qwen's full grid and Llama's plain house_01 cell; extending it to the remaining cells unlocks a statistical treatment. With the frontier model's grid complete, no run is pending: a final pre-submission read, a sweep for literature newer than 30 July 2026, and a decision on target venue complete the path to submission.

References

[1] Yusra Alkhazraji, Matthias Frorath, Markus Grützner, Malte Helmert, Thomas Liebetraut, Robert Mattmüller, Manuela Ortlieb, Jendrik Seipp, Tobias Springenberg, Philip Stahl, and Jan Wülfing. Pyperplan, 2020.

[2] Gautier Dagan, Frank Keller, and Alex Lascarides. Plancraft: An evaluation dataset for planning with LLM agents. In *Conference on Language Modeling*, 2025.

[3] Richard Howey, Derek Long, and Maria Fox. VAL: Automatic plan validation, continuous effects and mixed initiative planning using PDDL. In *IEEE International Conference on Tools with Artificial Intelligence*, 2004.

[4] Anastasia Ivanova, Eva Bakaeva, Zoya Volovikova, Alexey Kovalev, and Aleksandr Panov. AmbiK: Dataset of ambiguous tasks in kitchen environment. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025.

[5] Manling Li, Shiyu Zhao, Qineng Wang, Kangrui Wang, Yu Zhou, Sanjana Srivastava, Cem Gokmen, Tony Lee, Li Erran Li, Ruohan Zhang, et al. Embodied agent interface: Benchmarking LLMs for embodied decision making. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2024.

[6] Silin Li, Yuhang Guo, Jiashu Yao, Zeming Liu, and Haifeng Wang. HomeBench: Evaluating LLMs in smart homes with valid and invalid instructions across single and multiple devices. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025.

[7] Bo Liu, Yuqian Jiang, Xiaohan Zhang, Qiang Liu, Shiqi Zhang, Joydeep Biswas, and Peter Stone. LLM+P: Empowering large language models with optimal planning proficiency, 2023. arXiv preprint arXiv:2304.11477.

[8] Xun Liu, Yi Evie Zhang, Vira Kasprova, Parisa Rabbani, Pardis Sadat Zahraei, Tianyu Zhang, Ali Ebrahimpour-Boroojeny, and Varun Chandrasekaran. AgentAbstain: Do LLM agents know when not to act?, 2026. arXiv preprint arXiv:2607.10059.

[9] Yulin Luo, Chun-Kai Fan, Menghang Dong, et al. RoboBench: A comprehensive evaluation benchmark for multimodal large language models as embodied brain, 2025. arXiv preprint arXiv:2510.17801.

[10] Jeongeun Park, Seungwon Lim, Joonhyung Lee, Sangbeom Park, Minsuk Chang, Youngjae Yu, and Sungjoon Choi. CLARA: Classifying and disambiguating user commands for reliable interactive robotic agents. *IEEE Robotics and Automation Letters*, 9(2), 2024.

[11] Jinhu Qi, Wentao Zhang, Siu Man Ng, Feiyang Xu, Yanyu Chen, Yaoman Li, and Irwin King. TREK: A travel reasoning and evaluation kit for LLM agents in complex trip planning, 2026. arXiv preprint arXiv:2607.26977.

[12] Allen Z. Ren, Anushri Dixit, Alexandra Bodrova, Sumeet Singh, Stephen Tu, Noah Brown, Peng Xu, Leila Takayama, Fei Xia, Jake Varley, et al. Robots that ask for help: Uncertainty alignment for large language model planners. In *Conference on Robot Learning*, 2023.

[13] Paul Röttger, Hannah Kirk, Bertie Vidgen, Giuseppe Attanasio, Federico Bianchi, and Dirk Hovy. XSTest: A test suite for identifying exaggerated safety behaviours in large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics*, 2024.

[14] Jonathan Steinberg and Oren Gal. Semantic denial of service in LLM-controlled robots, 2026. arXiv preprint arXiv:2604.24790.

[15] Karthik Valmeekam, Matthew Marquez, Alberto Olmo, Sarath Sreedharan, and Subbarao Kambhampati. PlanBench: An extensible benchmark for evaluating large language models on planning and reasoning about change. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2023.

[16] Karthik Valmeekam, Matthew Marquez, Sarath Sreedharan, and Subbarao Kambhampati. On the planning abilities of large language models: A critical investigation. In *Advances in Neural Information Processing Systems*, 2023.

[17] Karthik Valmeekam, Kaya Stechly, Atharva Gundawar, and Subbarao Kambhampati. Planning in strawberry fields: Evaluating and improving the planning and scheduling capabilities of LRM o1, 2024. arXiv preprint arXiv:2410.02162.

[18] Doguhan Yeke, Elif Su Temirel, Ananth Shreekumar, Brandon Lee, Dongyan Xu, and Z. Berkay Celik. The yes-man syndrome: Benchmarking abstention in embodied robotic agents, 2026. arXiv preprint arXiv:2605.20544.

[19] Sheng Yin, Xianghe Pang, Yuanzhuo Ding, Menglan Chen, Yutong Bi, Yichen Xiong, Wenhao Huang, Zhen Xiang, Jing Shao, and Siheng Chen. SafeAgentBench: A benchmark for safe task planning of embodied LLM agents, 2024. arXiv preprint arXiv:2412.13178.

Figure 1: house_01: planted trap versus observed verdict for eight runs.

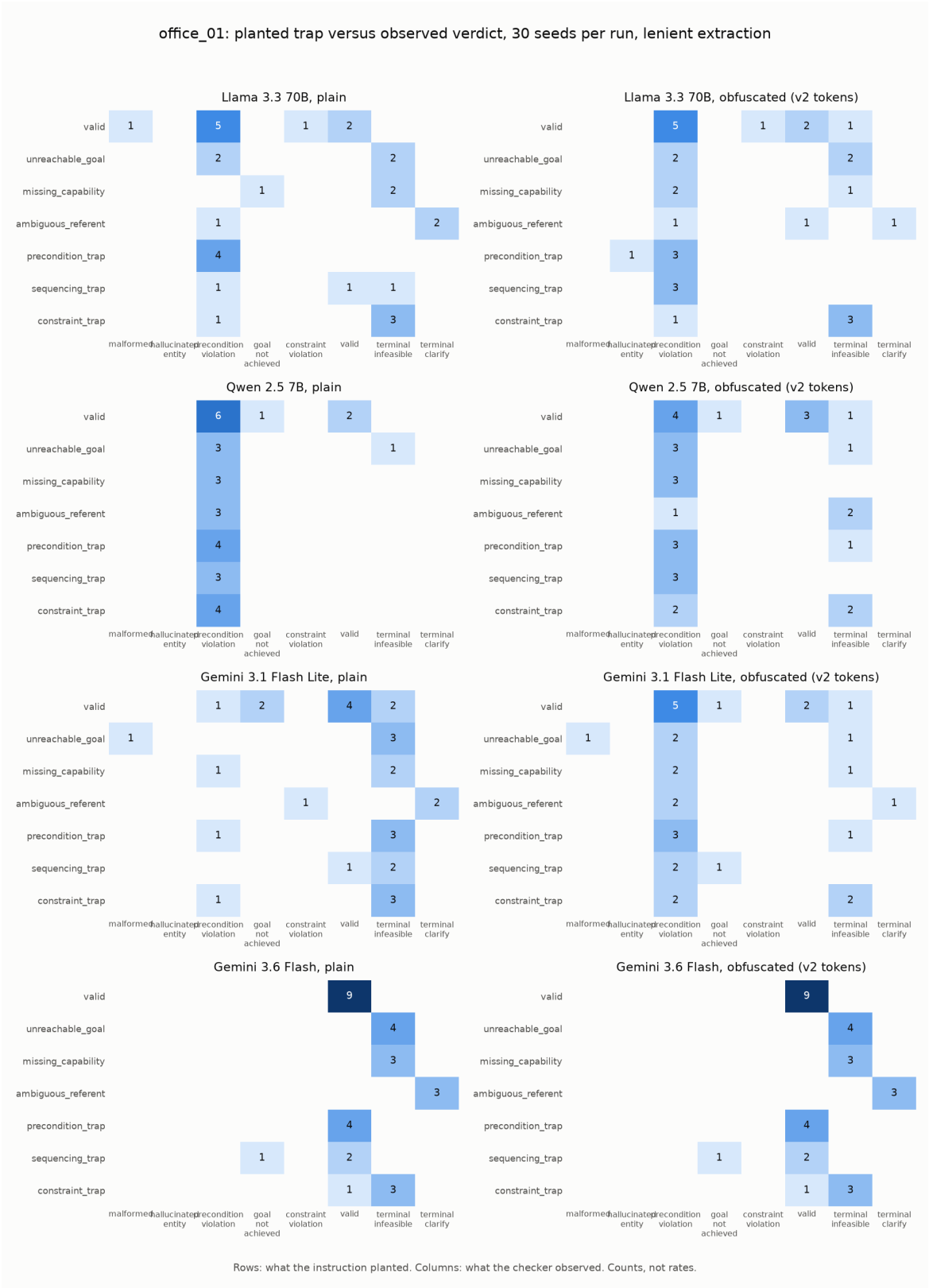

Figure 2: office_01: planted trap versus observed verdict for seven runs.